Simulation and Detection of SSH-Based Botnet Attacks

Anna Poglitsch up202511830@edu.fe.up.pt
David Azevedo da Silva up202104721@edu.fe.up.pt
Rúben Jordão Oliveira up202205106@edu.fe.up.pt
Wendell Gustavo Ferreira Pinto up202200049@edu.fe.up.pt

***Abstract*—The Mirai malware is known for some of the largest distributed denial-of-service (DDoS) attacks. It infects Linux-based devices and turns them into remotely controlled bots for large-scale network attacks. This project simulates the full lifecycle of an SSH-based botnet attack — reconnaissance, credential brute-forcing, lateral movement through segmented networks, and command-and-control communication — in a controlled, containerised environment. A key contribution is a real-time browser dashboard that visualises the live attack topology via Server-Sent Events, integrates scenario setup, and enables live application of defenses so their effect can be observed as the botnet runs. Four defensive countermeasures are evaluated — Fail2Ban, firewall IP blocking, SSH rate limiting, and disabling password authentication — across four network scenarios of increasing complexity, demonstrating that layered defenses are necessary to prevent multi-stage compromise and lateral movement. All code, configuration, and instructions to reproduce the experiments are at https://github.com/wendellgust/ssh-botnet-lab.**

I. Introduction

Context. Cyberattacks increasingly involve multi-stage techniques combining information gathering, credential attacks, lateral movement, and command-and-control (C2) communication. The SSH protocol, while secure by design, is frequently misconfigured in production environments — particularly through enabled password authentication — making it a common entry point for automated botnet propagation, as demonstrated by malware families such as Mirai [1], [2].

Problem. Traditional cybersecurity courses focus on isolated attack techniques without demonstrating the complete attack lifecycle. Real-world intrusions involve sequential phases, and the interaction between those phases and defensive countermeasures is not well understood in educational settings.

Goals. This project addresses three questions:

1) Can a Python-based botnet autonomously propagate across segmented networks using only SSH brute-force and pivoting?
2) How effective are common SSH hardening measures (Fail2Ban, IP blocking, rate limiting, key-only auth) individually and in combination?
3) Does attack depth (number of pivot hops) affect the relative effectiveness of each defense?

Unlike existing SSH security exercises that test individual techniques in isolation, this project contributes an integrated environment covering the complete botnet lifecycle — initial brute-force, multi-hop lateral movement, honeypot evasion, and C2 beaconing — together with a real-time browser dashboard that lets an operator observe the attack progressing live, apply defenses mid-run, and immediately see their effect on the network topology. The lab runs entirely in rootless Podman containers with no internet access. All code and instructions to reproduce the experiments are at https://github.com/wendellgust/ssh-botnet-lab.

II. Related Work

The Mirai botnet became widely known after launching large-scale DDoS attacks in 2016 [1]. A comprehensive post-mortem by Antonakakis et al. [2] characterised its scanning pipeline — TCP SYN sweep, credential brute-force, and C2 enrolment — which directly inspired our botnet's attack phases. Mirai demonstrated that weak default SSH credentials are sufficient to compromise devices at scale without exploiting any software vulnerability.

The SSH protocol was introduced by Ylönen [3] as a cryptographically secure remote shell; yet password authentication — retained for backward compatibility — remains a persistent misconfiguration in production environments [4]. Our lab reproduces this exact condition: all victim containers have password authentication enabled with weak credentials, mirroring real-world deployments.

Honeypot-based detection [5] provides high-confidence alerts because legitimate traffic never reaches a honeypot. We adopt this principle directly: our honeypot container is reachable only by traversing an internal network segment, so any connection to it is definitive evidence of successful lateral movement.

Network segmentation and defense-in-depth [6], [7] are the primary architectural countermeasures evaluated. Our multi-scenario design tests whether segmentation alone is sufficient and at what point combined controls become necessary — a question the literature addresses theoretically but rarely validates empirically in a reproducible lab environment.

III. Project Description

A. Lab Architecture

The lab consists of Podman-based containerised hosts running Ubuntu 22.04, connected through isolated bridge networks with no internet access. Five container roles cooperate to simulate the full attack and defense lifecycle:

- **Attacker:** Python botnet (Paramiko) that sequentially scans for live SSH hosts, brute-forces credentials from a wordlist with configurable inter-attempt delay, installs a

relay on compromised dual-homed hosts, and propagates laterally through each discovered network segment.
- **Victims:** Ubuntu containers running real OpenSSH server with intentionally weak credentials (`labuser` with common passwords). Every authentication attempt — successful or failed — is recorded in the container's `/var/log/auth.log`, producing authentic forensic evidence.
- **Honeypot:** Mimics a legitimate internal host but logs every inbound connection with timestamp, source IP, and protocol. Because no legitimate traffic should ever reach it, any connection is a confirmed indicator of successful lateral movement.
- **Monitor:** Custom Python detection engine that tails auth logs across all containers and evaluates three IDS rules: (1) brute-force burst — more than 5 failed attempts from the same source within 60 s; (2) honeypot trigger — any inbound connection logged by the honeypot; (3) C2 beacon — TCP connections to the attacker's listener port (8888) from inside the network. Each fired rule produces a human-readable alert with the indicator, evidence, and recommended response action.
- **GUI server:** Custom Python HTTP/SSE server (no external framework) providing a real-time browser dashboard at `http://localhost:5000`.

B. Real-Time GUI

The GUI is a central contribution of the lab. It serves a live network topology diagram rendered in SVG, updated in real time via Server-Sent Events (SSE) as the botnet progresses. Nodes change colour to reflect state transitions — idle, scanning, brute-forcing, compromised — and animated packet dots travel along edges to visualise active connections.

Beyond visualisation, the GUI integrates the full lab workflow. A scenario selector (S1–S4) triggers container setup via `auto_run.sh` and streams build output to the log feed; the *Run Botnet* button activates only after setup completes. A *Defenses* panel allows applying any of the four countermeasures to any victim with a single click, by executing the corresponding iptables, Fail2Ban, or sshd commands inside the target container via `podman exec`. This enables live, mid-run defense application — an operator can observe the attack stalling in real time as a defense takes effect. An attack report panel summarises discovered hosts, credentials found, pivot paths, and IDS alerts at any point during or after a run.

C. Scenarios

Four scenarios of increasing complexity are defined (see Appendix A for topology diagrams):

- **S1 — Flat:** Single network (172.21.0.0/24). All hosts directly reachable.
- **S2 — Single pivot:** Two networks. Attacker must compromise Victim 1 (dual-homed) to reach internal_net (10.10.0.0/24).
- **S3 — Parallel pivots:** Three networks. Two independent lateral movement paths via Victim 1 and Victim 2.
- **S4 — Deep chain:** Three networks. Two-hop chain: attacker → Victim 1 → Victim 3 → deep_net (10.30.0.0/24).

D. Attack Phases

Each scenario executes the following phases automatically:

1) **Scan:** The botnet iterates over known IP ranges for the current network segment, attempting an SSH connection on port 22 to identify live hosts.
2) **Brute-force:** For each live host, the botnet tries all credential pairs from the wordlist via Paramiko. A configurable delay between attempts controls attack speed and influences whether rate-based defenses trigger.
3) **Pivot preparation:** Once a dual-homed host is compromised, the botnet opens a Paramiko `direct-tcpip` channel through the host's existing SSH daemon — a standard SSH port-forwarding mechanism. This creates a transparent TCP tunnel to the next network segment without writing any files to the pivot host. From the perspective of internal hosts, SSH connections arrive from the pivot's internal IP, not from the external attacker.
4) **Lateral movement:** The botnet tunnels new SSH connections through the relay to reach hosts on internal segments that are otherwise unreachable from the attacker. The same scan-and-brute-force logic repeats for each discovered segment.
5) **Honeypot trigger:** From inside the compromised network, the botnet attempts a connection to the honeypot IP. This step intentionally produces a detectable event to test IDS alerting for lateral movement.
6) **C2 beaconing:** Each compromised bot establishes a TCP connection to the attacker's listener on port 8888 and sends five JSON heartbeat messages identifying itself. Bots in deeper segments relay their beacons through the chain of pivot hosts, demonstrating multi-hop C2 communication.

E. Defenses

Four countermeasures are applied to Victim 1 (the primary target and pivot host) and evaluated independently and in combination. Each defense is enforced by executing the corresponding command inside the container via `podman exec` — no container restart required.

- **Fail2Ban:** Monitors `/var/log/auth.log` and inserts a temporary iptables DROP rule for the attacker's IP after 3 failed SSH attempts within 60 s. Reactive — requires failed attempts to trigger; the attacker may succeed before the ban fires if the first credential in the wordlist is correct.
- **Block IP:** Inserts a permanent iptables DROP rule for the attacker's source IP (172.21.0.10). Proactive and absolute — all packets from that IP are silently dropped at the kernel level, regardless of SSH configuration. Ineffective against spoofed source IPs or lateral movement originating from a different host.

- **Rate limit:** Uses iptables conntrack to allow at most 10 new SSH connections per 60 s. Does not block the attacker — it slows attempts to a rate that may extend the engagement window past a practical threshold, but a sufficiently patient attacker can still brute-force successfully.
- **Disable password:** Appends `PasswordAuthentication no` to `/etc/ssh/sshd_config` and reloads sshd. Eliminates the brute-force vector entirely on the protected host regardless of password strength or wordlist size; the only remaining attack path is public-key compromise, which is outside this threat model.

F. Replication

The full environment is reproducible from the repository. A Linux host with Podman is the only dependency. Running `bash scripts/auto_run.sh 2` builds all images, starts the containers, executes the full attack chain, and prints a detection report.

IV. Assessment

A. Methodology

All data was collected using `scripts/defense_test.sh`, an automated script that cycles through all seven defense configurations for a given scenario. For each configuration it: (1) restarts all victim containers to clear iptables rules, Fail2Ban state, and auth logs; (2) applies the defense(s) exclusively to Victim 1 via `podman exec`; (3) triggers the full attack via the GUI `/run` endpoint, which executes `botnet.py` inside the attacker container; (4) waits for the botnet to finish and collects `/var/log/auth.log` from every victim, iptables state, and a structured JSON report. The primary metrics are number of compromised hosts and whether the attacker successfully pivoted into internal networks. Since defenses are applied only to Victim 1, other hosts on the attack network remain undefended throughout all configurations; a residual compromise count of 1 in some rows of Table I reflects these undefended hosts, not a bypass of the applied defense.

B. Results

Table I summarises results across all four scenarios and seven configurations. Full charts are in Appendix A.

Goal 1 — Autonomous propagation. Confirmed in all baseline runs. The botnet discovered live hosts, brute-forced credentials, prepared pivot relays, and propagated to internal segments without any manual intervention. In S2 and S3 the honeypot was triggered from inside the internal network segment, confirming successful lateral movement. In S4, eight hosts were compromised across three network layers via a two-hop chain — demonstrating that the botnet correctly tunnelled its scanning and brute-force logic through two consecutive relay nodes before reaching the deepest segment.

Goal 2 — Defense effectiveness. No single defense stops all compromise. Rate limiting is the strongest standalone control in S1, S2, and S4, reducing compromised hosts to zero and preventing pivoting. Fail2Ban is equally effective in S2 and S4. Disabling password authentication fully eliminates brute-force against the protected host — credential attempts become impossible — but undefended hosts on the same segment remain reachable. IP blocking has the same scope limitation. Layered defense (Fail2Ban + rate limit) stops all propagation in every scenario, including S3 with its two independent pivot paths.

Goal 3 — Effect of attack depth. Depth amplifies the failure of single-control defenses. In S3 (two parallel pivots), IP blocking and disable-password configurations yield four compromised hosts — unchanged from baseline — because the unblocked pivot path via Victim 2 remains fully operational. A notable anomaly in S3 is that rate limiting results in *more* compromised hosts (6) than the baseline (4). This occurs because rate limiting slows but does not block brute-force on Victim 1's path, extending the total run time. The extra time allows the botnet to fully traverse the second pivot path through Victim 2 and compromise additional hosts on extra_net that were not reached during the faster baseline run. This finding highlights that a defense which reduces attack speed without blocking it entirely can paradoxically increase the attack surface explored over a fixed engagement window. In S4, the opposite pattern holds: because the entire chain depends on Victim 1 as the sole first hop, rate limiting at the perimeter collapses the full attack path even without defending internal hosts.

C2 detection. The monitor detected 10 C2 beacon events per run in S2–S4, with bot IDs from both direct and relayed hosts. Three IDS rules fired consistently: the brute-force burst rule triggered within the first 60 s of every baseline run; the honeypot rule fired upon lateral entry into the internal segment; and the C2 rule identified beacon traffic tunnelled through pivot relays. Each alert included a specific response recommendation — isolate the pivot host, block east-west SSH forwarding, or inspect honeypot event logs — demonstrating that the detection engine can guide an operator through the full incident response sequence.

C. Limitations

The credential wordlist is small and targeted; larger dictionaries or credential-stuffing lists would alter Fail2Ban thresholds. All containers run on a single physical host, which may compress attack durations. The botnet uses a fixed inter-attempt delay; adaptive scanning could evade rate-limiting. The C2 channel is unencrypted; real botnets use encrypted or peer-to-peer C2 that is harder to detect.

V. Conclusion

This project demonstrated that SSH-based botnet propagation across segmented networks is feasible with minimal tooling when password authentication is enabled. Four defensive countermeasures were evaluated across four topologies of increasing complexity. No single defense is universally effective — rate limiting is the strongest standalone control,

TABLE I: Compromised hosts and pivot success across all scenarios and defense configurations.

2*Defense	Scenario 1		Scenario 2		Scenario 3		Scenario 4	
	Comp.	Pivot	Comp.	Pivot	Comp.	Pivot	Comp.	Pivot
Baseline	3	No	4	Yes	4	Yes	8	Yes
Fail2Ban	1	No	0	No	2	Yes	1	No
Block IP	1	No	1	No	4	Yes	1	No
Rate Limit	0	No	0	No	6	Yes	0	No
Disable Password	1	No	1	No	4	Yes	1	No
Fail2Ban + Rate Limit	0	No	0	No	0	No	0	No
All Defenses	1	No	0	No	0	No	1	No

but multi-path scenarios require layered defenses. Combining Fail2Ban with rate limiting consistently stopped propagation in all scenarios. The fully containerised, reproducible lab provides a reusable platform for studying attacker behaviour and evaluating defensive mechanisms. All artifacts are available at https://github.com/wendellgust/ssh-botnet-lab.

ACKNOWLEDGMENT

This work was carried out as part of the Security and Reliability of Systems (SSR) course at the Faculty of Engineering, University of Porto (FEUP).

REFERENCES

[1] J. Margolis, T. T. Oh, S. Jadhav, Y. H. Kim, and J. N. Kim, "An in-depth analysis of the Mirai botnet," in *2017 International Conference on Software Security and Assurance (ICSSA)*, 2017, pp. 6–12.

[2] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, 2017, pp. 1093–1110.

[3] T. Ylönen, "SSH — secure login connections over the Internet," in *Proceedings of the 6th USENIX Security Symposium*. USENIX Association, 1996, pp. 37–42.

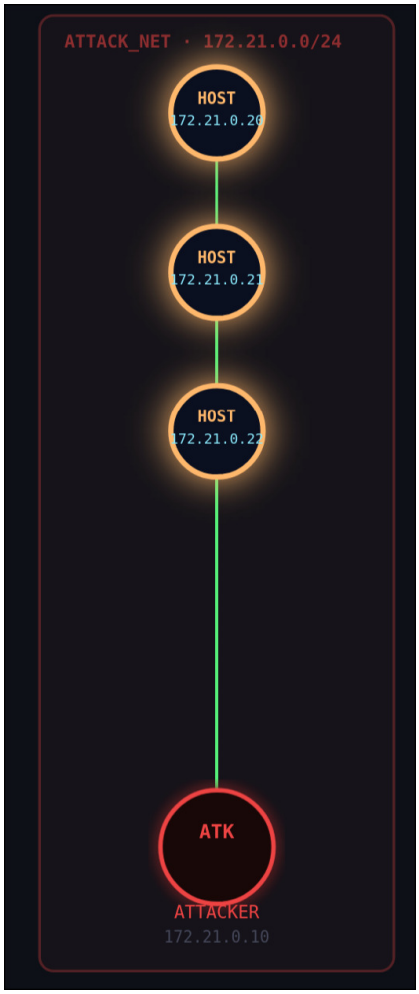
[4] E. Bou-Harb, M. Debbabi, and C. Assi, "Cyber scanning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 3, pp. 1496–1519, 2014.

[5] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2003.

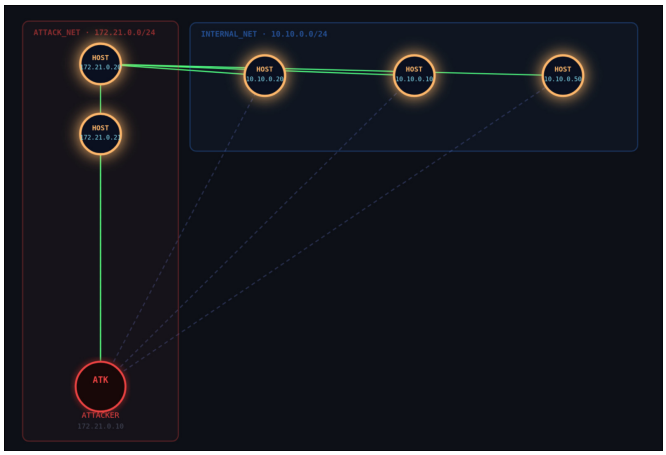
[6] National Institute of Standards and Technology, "Guide to general server security," NIST, Tech. Rep. Special Publication 800-123, 2008. [Online]. Available: https://csrc.nist.gov/publications/detail/sp/800-123/final

[7] M. Howard and D. LeBlanc, *Writing Secure Code*, 2nd ed. Microsoft Press, 2003.

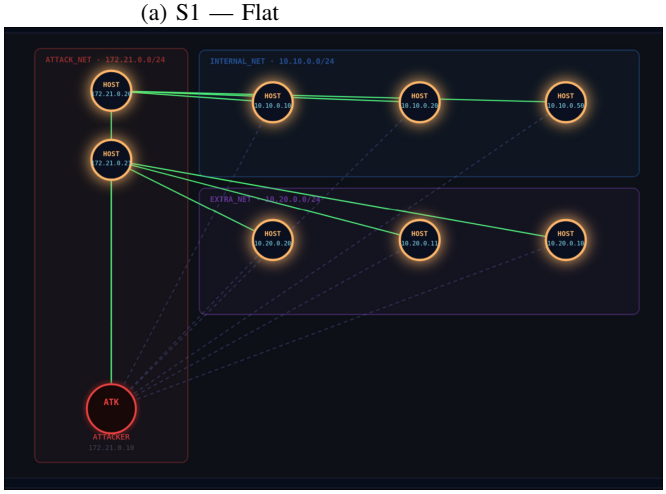
APPENDIX



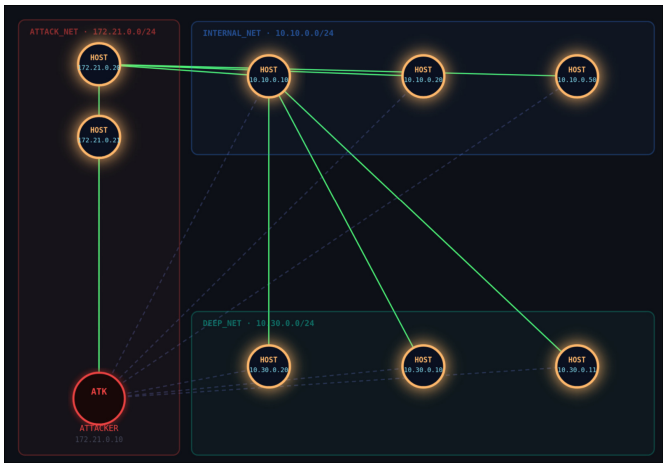
(a) S1 — Flat



(b) S2 — Single pivot

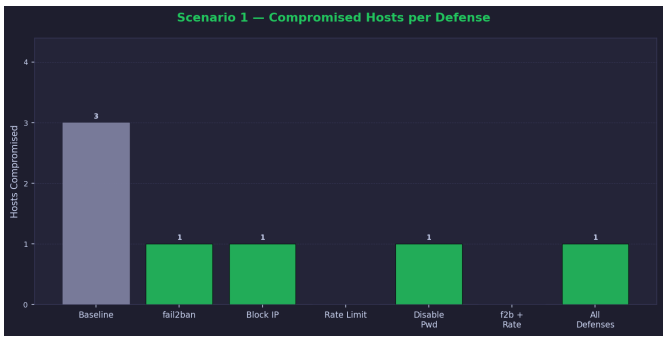


(c) S3 — Parallel pivots

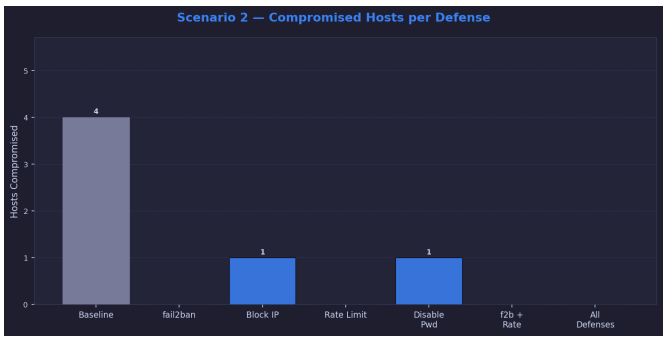


(d) S4 — Deep chain

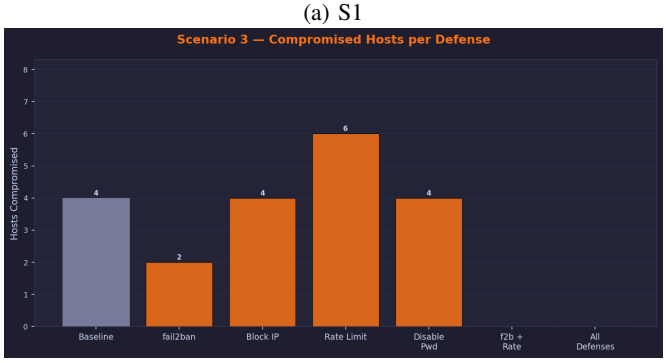
Fig. 1: Network topology for all four scenarios.



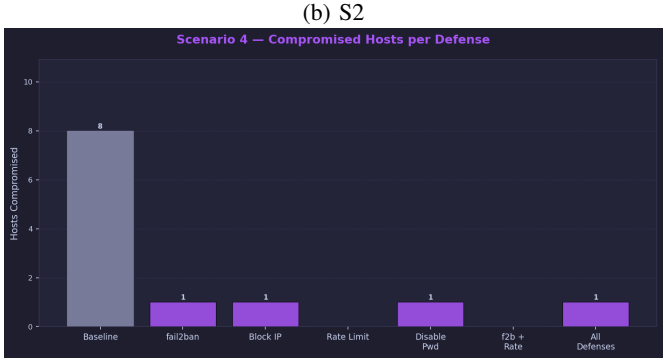
(a) S1



(b) S2

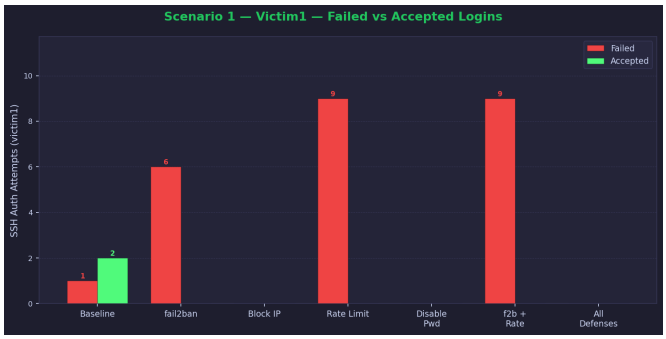


(c) S3

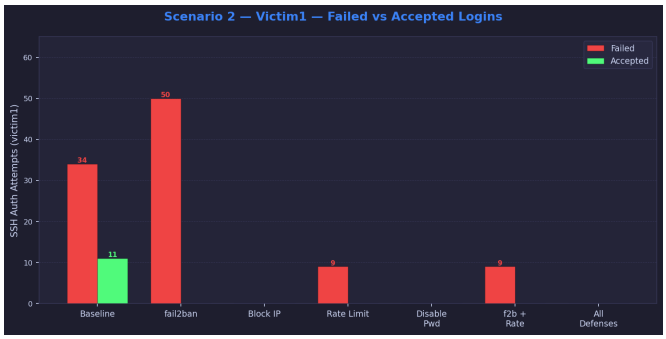


(d) S4

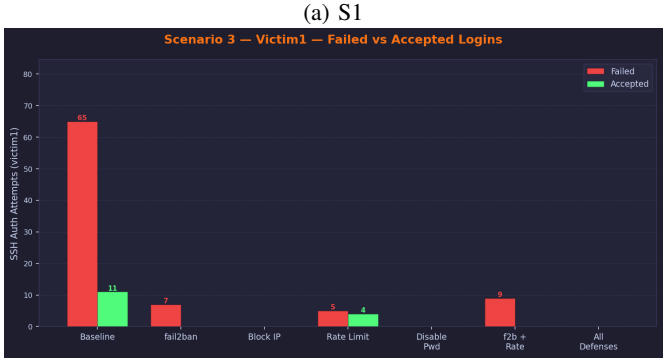
Fig. 2: Compromised hosts per defense configuration.



(a) S1



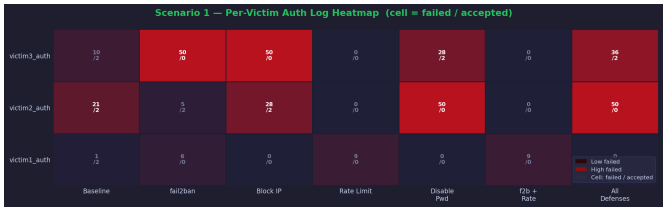
(b) S2



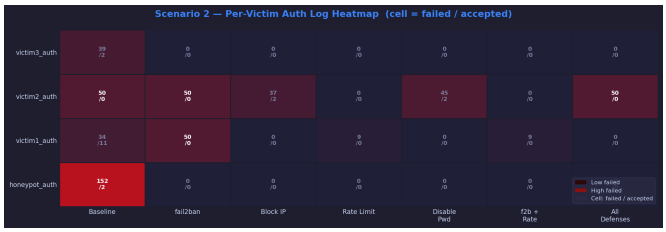
(c) S3

(d) S4

Fig. 3: Failed and accepted SSH authentication attempts per defense.



(a) S1

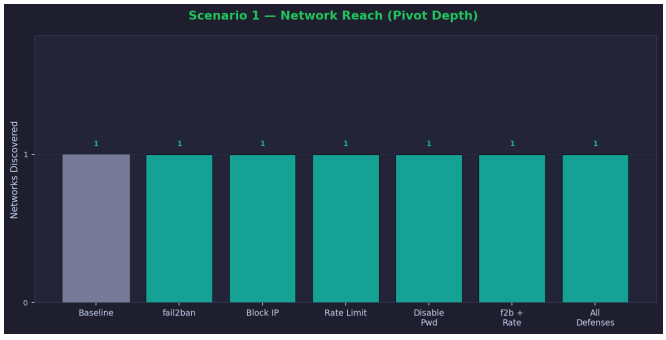


(b) S2

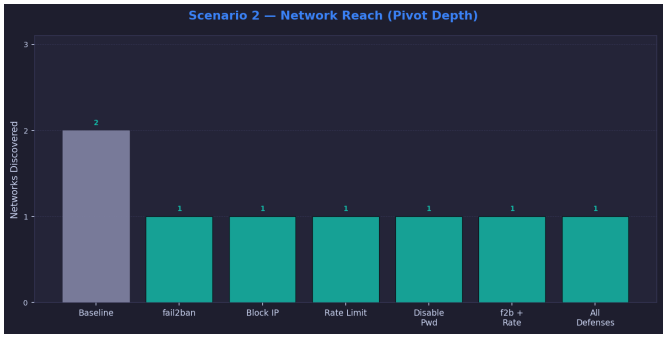
(c) S3

(d) S4

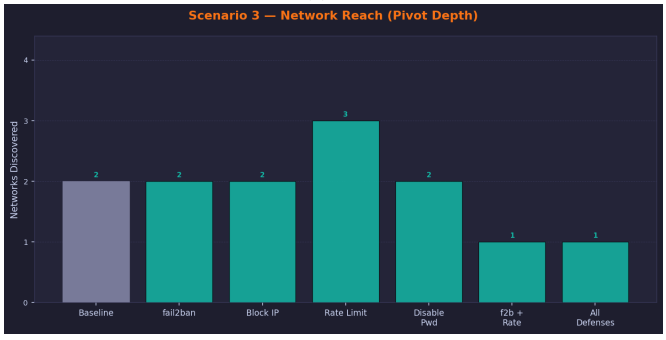
Fig. 4: Heatmap of brute-force activity across hosts and defense configurations.



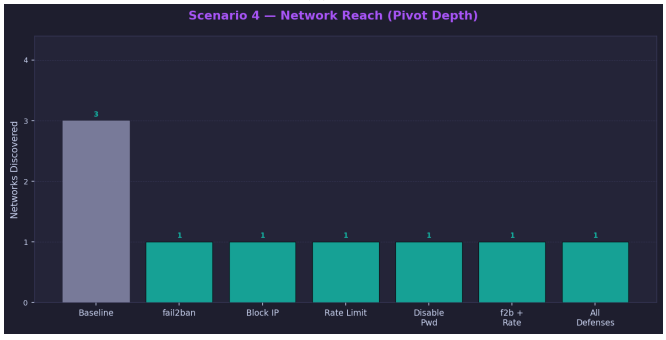
(a) S1



(b) S2

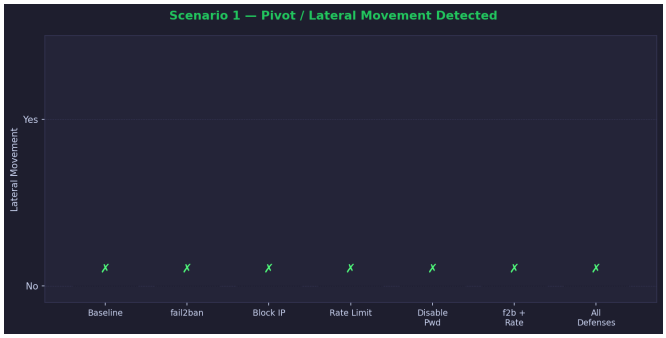


(c) S3



(d) S4

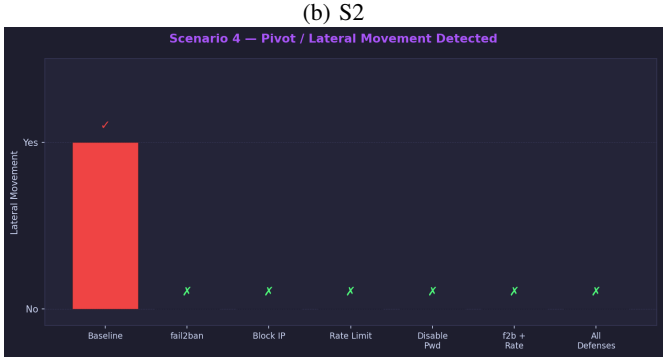
Fig. 5: Number of network segments reached per defense configuration.



(a) S1

(b) S2

(c) S3



(d) S4

Fig. 6: Pivot success per defense configuration.

Scenario 1 — Summary Table

Defense	v1 Failed	v1 Accepted	Compromised	Networks	Pivoted
Baseline	1	2	3	1	NO
fail2ban	6	0	1	1	NO
Block IP	0	0	1	1	NO
Rate Limit	9	0	0	1	NO
Disable Pwd	0	0	1	1	NO
f2b + Rate	9	0	0	1	NO
All Defenses	0	0	1	1	NO

(a) S1

Scenario 2 — Summary Table

Defense	v1 Failed	v1 Accepted	Compromised	Networks	Pivoted
Baseline	34	11	4	2	YES
fail2ban	50	0	0	1	NO
Block IP	0	0	1	1	NO
Rate Limit	9	0	0	1	NO
Disable Pwd	0	0	1	1	NO
f2b + Rate	9	0	0	1	NO
All Defenses	0	0	0	1	NO

(b) S2

Scenario 3 — Summary Table

Defense	v1 Failed	v1 Accepted	Compromised	Networks	Pivoted
Baseline	65	11	4	2	YES
fail2ban	7	0	2	2	YES
Block IP	0	0	4	2	YES
Rate Limit	5	4	6	3	YES
Disable Pwd	0	0	4	2	YES
f2b + Rate	9	0	0	1	NO
All Defenses	0	0	0	1	NO

(c) S3

Scenario 4 — Summary Table

Defense	v1 Failed	v1 Accepted	Compromised	Networks	Pivoted
Baseline	60	18	8	3	YES
fail2ban	6	0	1	1	NO
Block IP	0	0	1	1	NO
Rate Limit	9	0	0	1	NO
Disable Pwd	0	0	1	1	NO
f2b + Rate	9	0	0	1	NO
All Defenses	0	0	1	1	NO

(d) S4

Fig. 7: Detailed per-host summary tables.